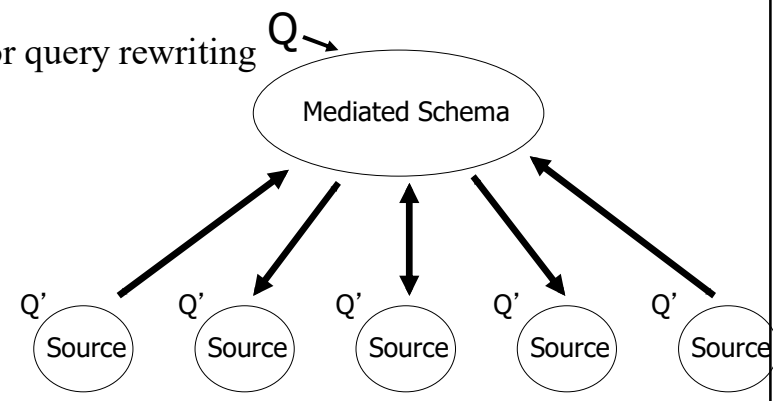
Schema Alignment and Query Rewriting

Vu Tuyet Trinh
SoICT - HUST

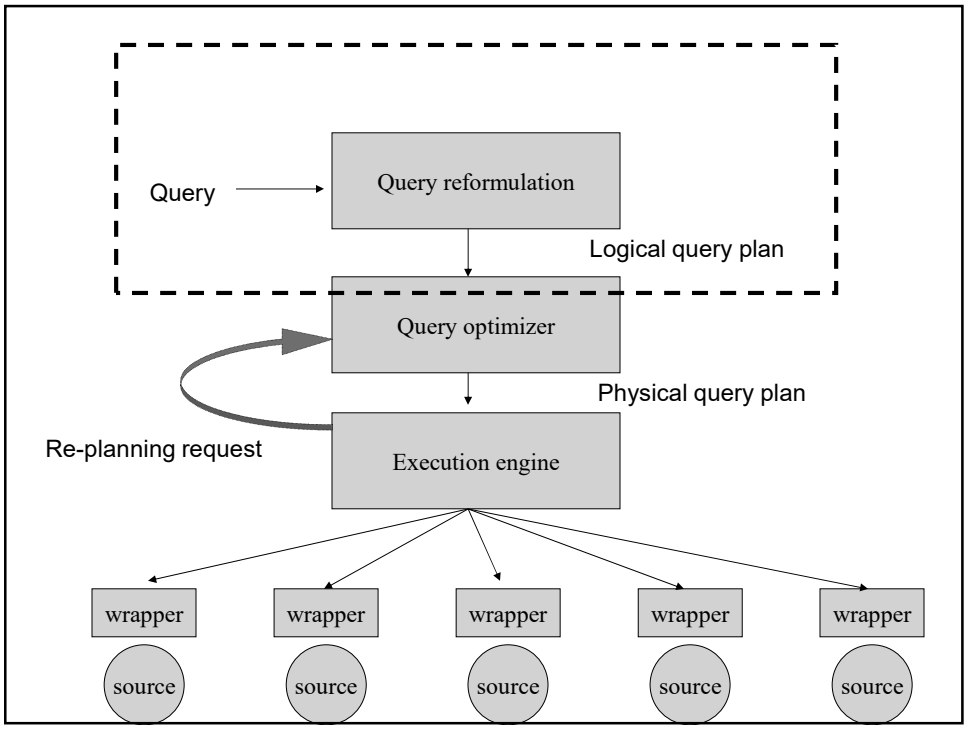
1

Schema Alignment / Schema matching

- Identifying correspondences between different data sources.
- Using for query rewriting



2



3

(Traditional) Data Integration

- Introduction to semantic heterogeneity
- Schema mapping languages
- Data-level heterogeneity
- Query unfolding

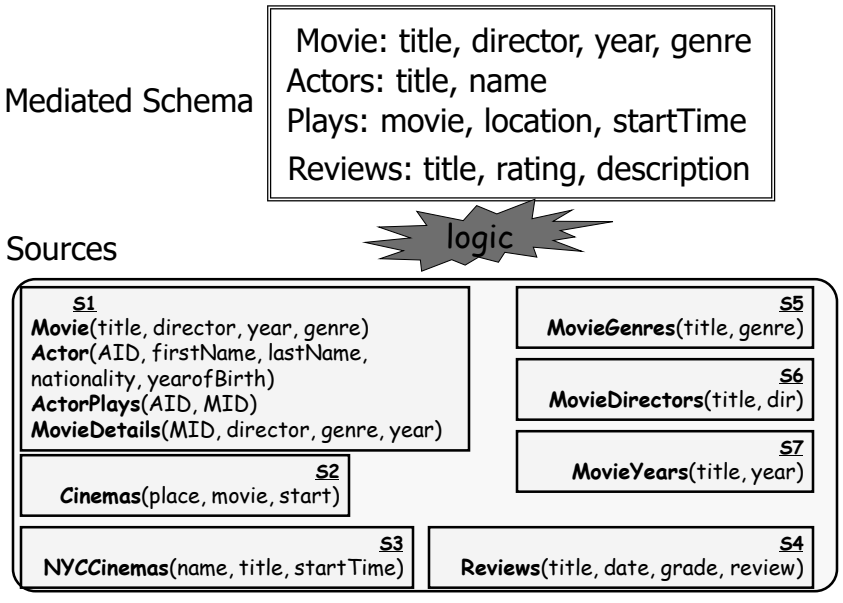
4

Schema Heterogeneity

- Schema heterogeneity is a fact of life.
  - Whenever schemas are designed by different people/organizations, they will be different, even if they model the *same* domain!
- The goal of schema mappings is to reconcile schema heterogeneity:
  - Mostly between the mediated schema and the schema of the data sources.

5

Schema Heterogeneity by Example



6

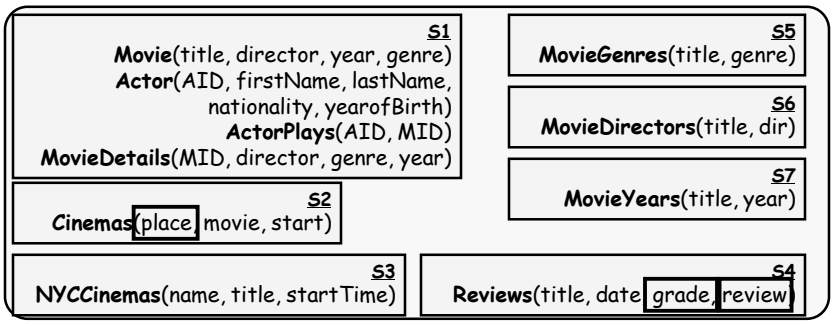
Table and Attribute Naming

Mediated Schema

Movie: title, director, year, genre
Actors: title, name
Plays: movie, location, startTime
Reviews: title, rating, description

Table and attribute names

Sources



7

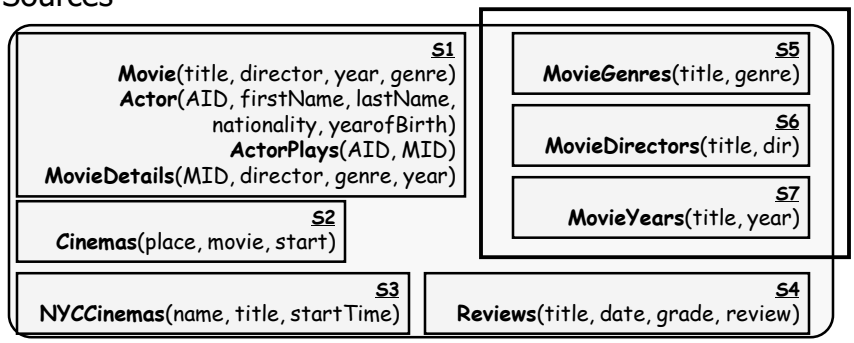
Tabular Organization of Schema

Mediated Schema

Movie: title, director, year, genre
Actors: title, name
Plays: movie, location, startTime
Reviews: title, rating, description

Different tabular organization

Sources



8

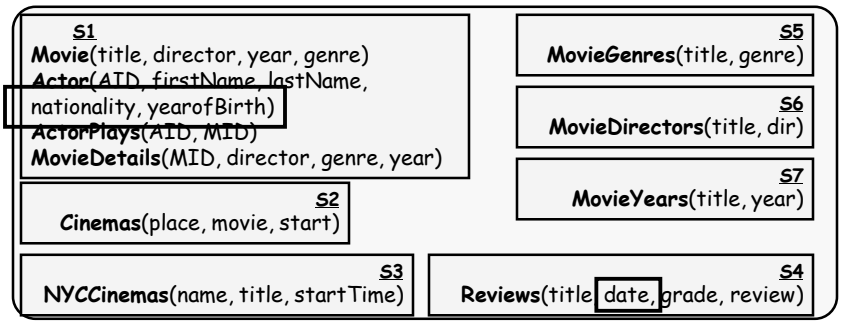
Schema Coverage

Mediated Schema

Movie: title, director, year, genre
Actors: title, name
Plays: movie, location, startTime
Reviews: title, rating, description

Different coverage and detail

Sources



9

Semantic Heterogeneity Summary

- Differences in:
  - Naming of schema elements
  - Organization of tables
  - Coverage and detail of schema
  - Data-level representation (IBM vs. International Business Machines)
- Reason:
  - Schemas probably designed for different applications/contexts.

10

More in Source Descriptions

- Possible access patterns to the data source:
  - Are there required inputs? (e.g., web forms, web services)
  - Can the source process complex database queries?
- Source completeness:
  - Is the source complete, or partially complete?
- Reliability, load restrictions, mirror sites, …

11

Outline

✓Introduction to semantic heterogeneity

➢Schema mapping languages

- Data-level heterogeneity
- Query unfolding

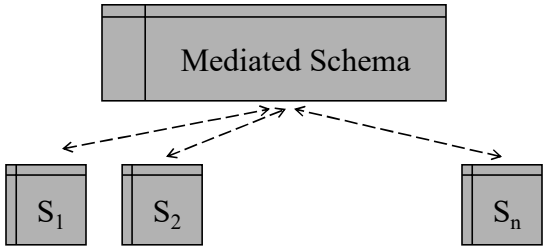
12

Principles of Schema Mappings

Schema mappings describe the relationship between:



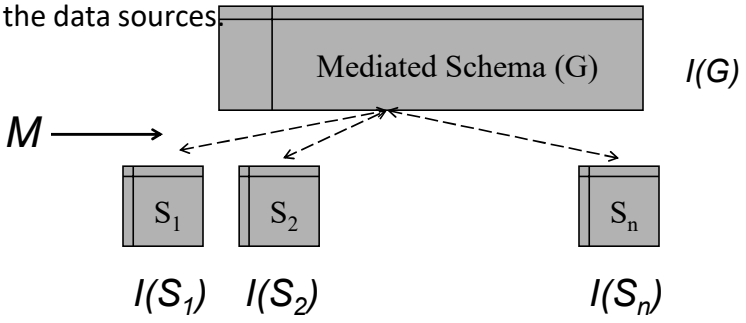
Or between:



13

Semantics of Schema Mappings

Formally, schema mappings describe a relation: which instances of the mediated schema are consistent with the current instances of the data sources.



I(G) (I(S_i)): the set of possible instances of the schema *G* (S_i).

$$M_R \subseteq I(G) \times I(S_1) \times \ldots I(S_n)$$

14

Relations, explained

- A relation is a subset of the Cartesian product of its columns' domains:

1	1
2	4
3	9
4	16
5	25

The table on the left is a subset of the Cartesian product of

Int x Int.

The table describes the Squared relation.

Simialry, M_r specifies the possible instances of the mediated schema, given instances of the sources.

15

Possible Instances of Mediated Schema: Simple Example

- Source 1: (Director, Title, Year) with tuples
  - {(Allen, Manhattan, 1979),
  - (Coppola, GodFather, 1972)}
- Mediated schema: (Title, Year)
  - Simple projection of Source 1
  - Only one possible instance:
  - {(Manhattan, 1979), (GodFather, 1972)}

16

Possible Instances of Mediated Schema: Second Example

- Source 1: (Title, Year) with tuples
  - {(Manhattan, 1979), (GodFather, 1972)}
- Mediated schema: (Director, Title, Year)
  - Possible instance 1: {(Allen, Manhattan, 1979), (Coppola, GodFather, 1972)}
  - Another possible instance 2: {(Halevy, Manhattan, 1979), (Stonebraker, GodFather, 1972)}.
- This matters when we answer queries:
  - See next slide.

17

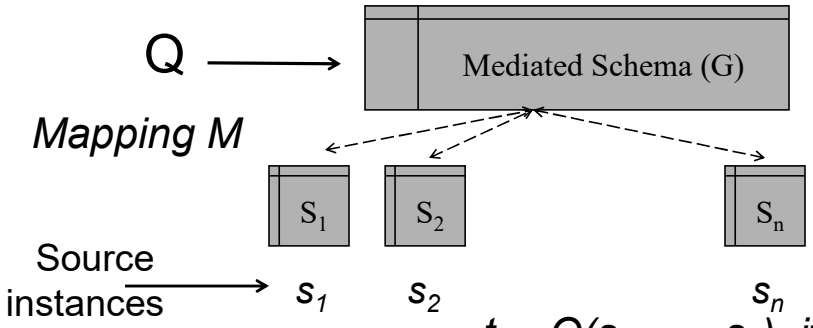
Answering Queries over Possible Instances of Mediated Schema

- Mediated schema: (Director, Title, Year)
  - Possible instance 1: {(Allen, Manhattan, 1979), (Coppola, GodFather, 1972)}
  - Another possible instance 2: {(Halevy, Manhattan, 1979), (Stonebraker, GodFather, 1972)}.
- Query Q1: return all years of movies
  - Answer: (1979, 1972) are certain answers.
- Query Q2: return all directors
  - No certain answers because no directors appear in all possible instances of the mediated schema.

18

Certain Answers Makes this Formal

An answer is certain if it is true in every instance of the mediated schema that is consistent with: (1) the instances of the sources and (2) the mapping *M*.



$t \in Q(s_1, \ldots, s_n)$ *iff*

$t \in Q(g)$ *for* $\forall g$, *s.t.* $(g, s_1, \ldots s_n) \in M_R$

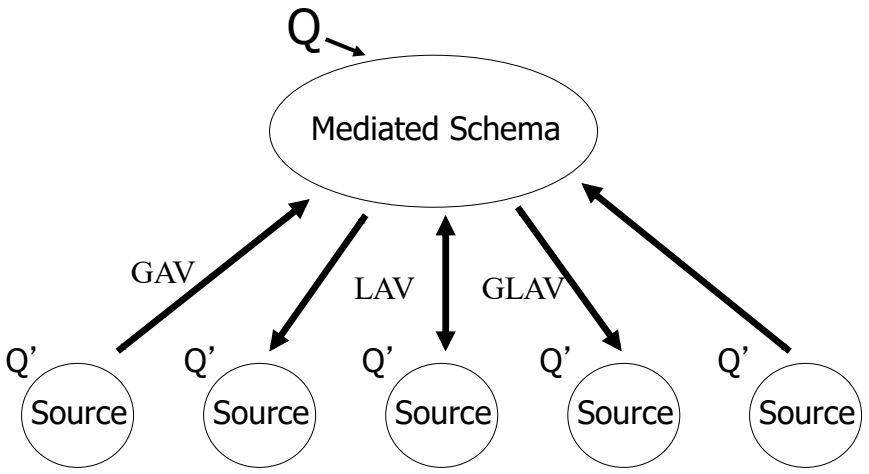
19

Desiderata from Source Description Languages

- Flexibility:
  - Should be able to express relationships between real schemata
- Efficient reformulation:
  - Computational complexity of reformulation and finding answers
- Easy update:
  - Should be easy to add and delete sources

20

Languages for Schema Mapping



21

Schema Heterogeneity by Example

Mediated Schema

Movie: title, director, year, genre
Actors: title, name
Plays: movie, location, startTime
Reviews: title, rating, description

logic

Sources

S1
Movie(title, director, year, genre)
Actor(AID, firstName, lastName, nationality, yearofBirth)
ActorPlays(AID, MID)
MovieDetails(MID, director, genre, year)

S2
Cinemas(place, movie, start)

S3
NYCCinemas(name, title, startTime)

S4
Reviews(title, date, grade, review)

S5
MovieGenres(title, genre)

S6
MovieDirectors(title, dir)

S7
MovieYears(title, year)

22

Global-as-View (GAV)

- Mediated schema defined as a set of views over the data sources

Movie: title, director, year, genre

S1
Movie(MID,title)
Actor(AID, firstName, lastName, nationality, yearofBirth)
ActorPlays(AID, MID)
MovieDetails(MID, director, genre, year)

$$Movie(title, director, year, genre) \supseteq$$
$$S1.Movie(MID, title),$$
$$S1.MovieDetail(MID, director, genre, year)$$

23

GAV: Formal Definition

A set of expressions of the form:

$$G_i(\bar{X}) \supseteq Q(\bar{S}) \quad \text{or} \quad G_i(\bar{X}) = Q(\bar{S})$$

open-world assumption

closed-world assumption

- G_i: relation in mediated schema
- $Q(\bar{S})$: query over source relations

24

GAV Example

Movie: title, director, year, genre

$$Movie(title, director, year, genre) \supseteq S1.Movie(MID, title), S1.MovieDetail(MID, director, genre, year)$$

$$Movie(title, director, year, genre) \supseteq S5.MovieGenres(title, genre), S6.MovieDirectors(title, director), S7.MovieYears(title, year)$$

25

GAV Example (cont.)

Plays: movie, location, startTime

$$Plays(movie, location, startTime) \supseteq S2.Cinemas(location, movie, startTime)$$

$$Plays(movie, location, startTime) \supseteq S3.NYCCinemas(location, movie, startTime)$$

S2
Cinemas(place, movie, start)

S3
NYCCinemas(name, title, startTime)

26

Reformulation in GAV

- Given a query Q on the mediated schema:
  - Return the best query possible on the data sources.

27

Reformulation in GAV = Query/View Unfolding

Q(title, location, startTime) :-
Movie(title, director, year, "comedy"),
Plays(title, location, st), st ≥ 8pm

$Movie(title, director, year, genre) \supseteq$

$S1.Movie(MID, title), S1.MovieDetail(MID, director, genre, year)$

$Plays(movie, location, startTime) \supseteq$

$S2.Cinemas(location, movie, startTime)$

28

First Reformulation

Q(title, location, startTime) :-
Movie(title, director, year, "comedy"),
Plays(title, location, st), st ≥ 8pm

Q'(title, location, startTime) :-
S1.Movie(MID, title),
S1.MovieDetail(MID, director, "comedy", year)
S2.Cinemas(location,title, st), st ≥ 8pm

29

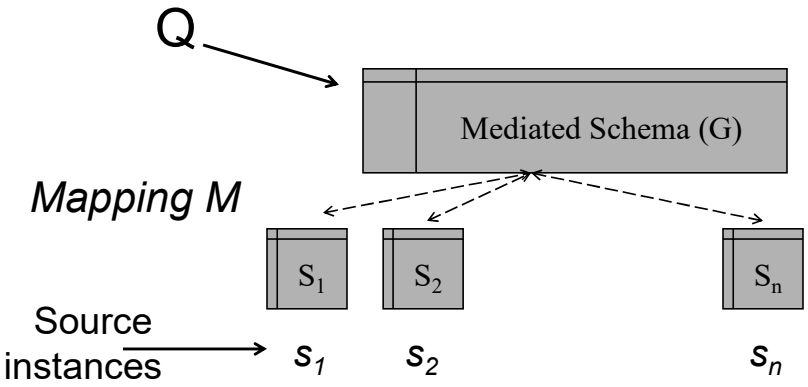
Another Reformulation

Q(title, location, startTime) :-
Movie(title, director, year, "comedy"),
Plays(title, location, st), st ≥ 8pm

Q'(title, location, startTime) :-
S1.Movie(MID, title),
S1.MovieDetail(MID, director, "comedy", year)
S3.NYCCinemas(location,title, st), st ≥ 8pm

30

Certain Answers (recall definition)

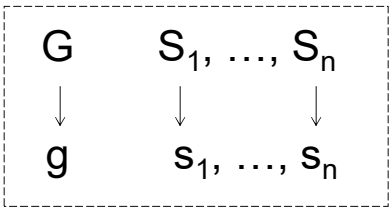


$t \in Q(s_1, \ldots, s_n)$ *iff*
$t \in Q(g)$ *for* $\forall g$, *s.t.* $(g, s_1, \ldots s_n) \in M_R$

31

Semantics of GAV

$(g, s_1, \ldots, s_n) \in M_R$ if:



$G_i(\overline{X}) \supseteq Q(\overline{S}) \quad \Rightarrow$

The extension of G_i in *g* is a super-set of evaluating Q_i on the sources.

$G_i(\overline{X}) = Q(\overline{S}) \quad \Rightarrow$

The extension of G_i in *g* is equal to evaluating Q_i on the sources.

32

Tricky Example for GAV

S8: stores pairs of (actor, director)

Movie: title, director, year, genre
Actors: title, name
Plays: movie, location, startTime
Reviews: title, rating, description

$$Actors(NULL, actor) \supseteq S8(actor, director)$$
$$Movie(NULL, director, NULL, NULL) \supseteq S8(actor, director)$$

33

Tricky Example for GAV

$$Actors(NULL, actor) \supseteq S8(actor, director)$$
$$Movie(NULL, director, NULL, NULL) \supseteq S8(actor, director)$$

Given the S8 tuples:
({Keaton, Allen}, {Pacino, Coppola})

We'd get tuples for the mediated schema:
Actors: ({NULL, Keaton}, {NULL, Pacino})
Movie: ({NULL, Allen, NULL, NULL},
{NULL, Coppola, NULL, NULL})

34

Tricky Example (2)

Actors: ({NULL, Keaton}, {NULL, Pacino})
Movie: ({NULL, Allen, NULL, NULL},
{NULL, Coppola, NULL, NULL})

Can't answer the query:
Q(actor, director) :-
Actors(title, actor),
Movie(title, director, genre, year)

LAV (Local as View) will solve this problem

35

GAV Summary

- Mediated schema is defined as views over the sources.
- Reformulation is conceptually easy
  - Polynomial-time reformulation and query answering.
- GAV forces everything into the mediated schema's perspective:
  - Cannot capture a variety of tabular organizations.

36

Local-as-View (LAV)

- Data sources defined as views over mediated schema!

S5 **MovieGenres**(title, genre)

S6 **MovieDirectors**(title, dir)

S7 **MovieYears**(title, year)

Movie: title, director, year, genre
Actors: title, name
Plays: movie, location, startTime
Reviews: title, rating, description

$$S5.MovieGenres(title,genre) \subseteq Movie(title,dir,year,genre)$$

$$S6.MovieDirectors(title,dir) \subseteq Movie(title,dir,year,genre)$$

37

Local-as-View (LAV)

- Data sources defined as views over mediated schema!

S8 **ActorDirectors**(actor,dir)

Movie: title, director, year, genre
Actors: title, name
Plays: movie, location, startTime
Reviews: title, rating, description

$$S8.ActorDirectors(actor,dir) \subseteq$$
$$Movie(title,dir,year,genre), Actor(title,actor)$$
$$year \geq 1980$$

38

LAV: Formal Definition

A set of expressions of the form:

$S_i(\overline{X}) \subseteq Q_i(G)$ or $S_i(\overline{X}) = Q_i(G)$

open-world assumption

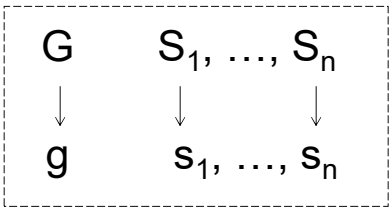
closed-world assumption

- $S_i(\overline{X})$: source relation
- $Q_i(G)$: query over mediated schema

39

Semantics of LAV

$(g, s_1, \ldots, s_n) \in M_R$ if:



$S_i(\overline{X}) \subseteq Q_i(G) \Rightarrow$

The result of Q_i over *g* is a superset of s_i.

$S_i(\overline{X}) = Q_i(G) \Rightarrow$

The result of Q_i over *g* equals s_i.

40

Possible Databases

Unlike GAV, LAV definitions imply a *set* of possible databases for the mediated schema.

$$S8.ActorDirectors(actor,dir) \subseteq$$
$$Movie(title,dir,year,genre), Actor(title,actor)$$

$$S8 : \{(Keaton, Allen)\}$$

Two possible databases for the mediate schema are:

Movie: {("manhattan", allen, 1979, comedy)}
Actor:{("manhattan", keaton)}

Movie: {("foobar", allen, 1979, comedy)}
Actor:{("foobar", keaton)}

41

Possible Databases

Since the source may be incomplete, other tuples may be in the instance of the mediated schema:

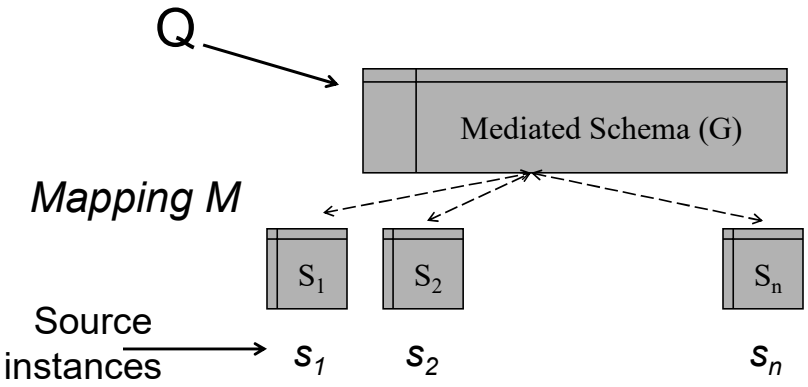
$$S8.ActorDirectors(actor,dir) \subseteq$$
$$Movie(title,dir,year,genre), Actor(title,actor)$$

$$S8 : \{(Keaton, Allen)\}$$

Movie: {(manhattan, allen, 1981, comedy),
(leatherheads,clooney,2008, comedy)}
Actor:{(manhattan, keaton),
(the godfather, keaton)}

42

Certain Answers: by now you know this slide by heart



$t \in Q(s_1, \ldots, s_n)$ *iff*
$t \in Q(g)$ *for* $\forall g$, *s.t.* $(g, s_1, \ldots s_n) \in M_R$

43

Certain Answers Example 1

$$S8.ActorDirectors(actor,dir) \subseteq$$
$$Movie(title,dir,year,genre), Actor(title,actor)$$
$$year \geq 1980$$

$$S8 : \{(Keaton, Allen)\}$$

$$Q(actor,dir) :-$$
$$Movie(title,dir,year,genre), Actor(title,actor)$$

Only one certain answer: *(Keaton, Allen)*

44

Certain Answers Example 2

$V_8(dir) = DirectorActor(ID, dir, actor)$ {Allen}

$V_9(actor) = DirectorActor(ID, dir, actor)$ {Keaton}

$Q(dir, actor) : -Director(ID, dir), Actor(ID, actor)$

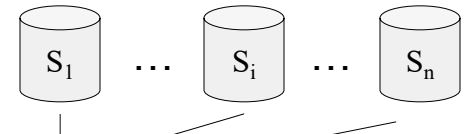
Under closed-world assumption:
single DB possible $\Rightarrow$ (Allen,Keaton)

Under open-world assumption:
no certain answers.

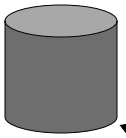
45

Reformulation in LAV

We're given tuples for sources (expressed as views)



Mediated schema (but no tuples)



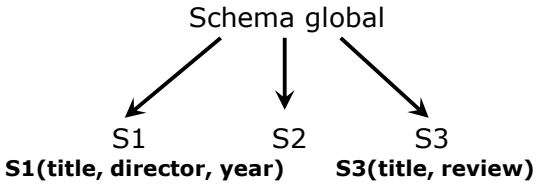
Query over mediated schema

This is exactly the problem of:…
Answering queries using views!

46

Example

MovieYear (title, year)
MovieRev(title, director, review)



S1: V1(title, year, director) :- MovieYear(title, year) & MovieRev(title, director, review)
S2: V2(title, year, director) :- MovieYear(title, year) & MovieRev(title, director, review)
S3: V3(title, review) :- MovieRev(title, director, review)

47

47

LaV - rewriting

- *Query*: look for reviews of films in 1997

q(title, review):- MovieYear(title, 1997) &
MovieReview(title, director, review)



q1(title, review):-V1(title, 1997,director) &
V3(title, review)
q2(title, review):-V2(title, 1997,director) &
V3(title, review)
...

48

48

Principle of rewriting in LaV

- Inverse rules

V1(title, year, director) :- MovieYear(title, year) &
MovieRev(title, director, review)

➔MovieYear(title, year) :- V1(title, year, _)

MovieRev(title, director,?) :-V1(title, _, director)

V3(title, review) :- MovieRev(title, _, review)

➔MovieRev(title, ?, review):-V3(title, review)

…

- Unfolding

q1(title, review):- V1(title, 1997, _) & V1(title, _, director)

q2(title, review):- V1(title, 1997, _) & V3(title,review)

…

- Cleaning

Check validity of unfolded queries

49

49

Local-as-View Summary

- Reformulation = answering queries using views
- Algorithms work well in practice:
  - Reformulation is not the bottleneck
- Under some conditions, guaranteed to find all certain answers
  - In practice, they typically do.
- LAV expresses incomplete information
  - GAV does not. Only a single instance of the mediated schema is consistent with sources.

50

LAV Limitation

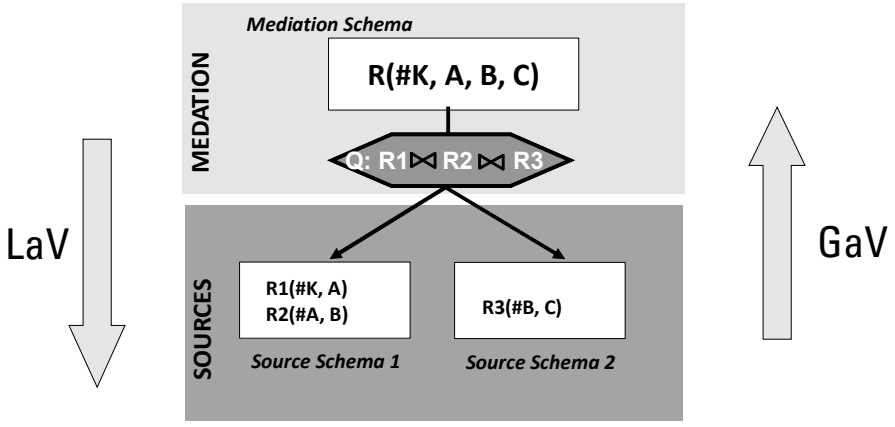
Movie: title, director, year, genre

S1
Movie(MID,title)
Actor(AID, firstName, lastName,nationality,yearofBirth)
ActorPlays(AID, MID)
MovieDetails(MID, director, genre, year)

If a key is internal to a data source, LAV cannot use it.

51

Comparaison: GaV vs. LaV



52

52

GLAV – a better solution ?

A set of expressions of the form:

$$Q^S(\overline{X}) \subseteq Q^G(\overline{X}) \text{ or } Q^S(\overline{X}) = Q^G(\overline{X})$$

- Q^G: query over mediated schema
  - Q^S: query over data sources

53

GLAV Example

$$S1.Movie(MID, title), S1.MovieDetail(MID, dir, genre, year)$$
$$\subseteq$$
$$Movie(title, dir, "comedy", year), year \geq 1970$$

54

Reformulation in GLAV

$$Q^S(\overline{X}) \subseteq Q^G(\overline{X})$$

- Given a query Q
- Find a rewriting Q' using the views

$$Q_1^G, \ldots, Q_n^G$$

- Create Q'' by replacing: $Q_i^G \rightarrow Q_i^S$
- Unfold $Q_1^S, \ldots, Q_n^S$

55

An Alternative Notation for GLAV: Tuple Generating Dependencies

Tuple generating dependencies (Chapter 2.1.2) can be used to specify GLAV expressions.

The TGD

$$(\forall \overline{X}) s_1(\overline{X}_1), \ldots, s_m(\overline{X}_m) \rightarrow (\exists \overline{Y})\, t_1(\overline{Y}_1), \ldots, t_k(\overline{Y}_k)$$

is equivalent to the GLAV expression: $Q^S(\overline{X}) \subseteq Q^G(\overline{X})$

where: $Q^S(\overline{X}) :- s_1(\overline{X}_1), \ldots, s_m(\overline{X}_m)$

$Q^G(\overline{Y}) :- t_1(\overline{Y}_1), \ldots, t_k(\overline{Y}_k)$

56

GLAV -> TGD Example

$$S1.Movie(MID,title), S1.MovieDetail(MID,dir,genre,year)$$
$$\subseteq$$
$$Movie(title,dir,"comedy",year), year \geq 1970$$

➔

$$S1.Movie(MID,title) \wedge S1.MovieDetail(MID,dir,genre,year)$$
$$\rightarrow Movie(title,dir,"comedy",year) \wedge year \geq 1970$$

Reformulation with TGD descriptions can be done relatively directly with the Inverse Rules Algorithm

57

Outline

- ✓Introduction to semantic heterogeneity
- ✓Schema mapping languages
- ➢Data-level heterogeneity
- Query unfolding

58

Data-Level Heterogeneity

- Huge problem in practice:
  - Data coming from different sources rarely joins perfectly.
- Differences of scale:
  - Celsius vs. Fahrenheit
  - Numeric vs. letter grades
  - First Name, Last Name vs. FullName
  - Prices with taxes or without
  - …

59

Mappings with Transformations

$$S(city, temp - 32 * 5/9, month) \subseteq Weather(city, temp, humidity, month)$$

$$CDStore(cd, price) \subseteq CDPrices(cd, state, price * (1 + rate)), LocalTaxes(state, rate)$$

60

Outline

✓Introduction to semantic heterogeneity
✓Schema mapping languages
✓Data-level heterogeneity
➢ Query unfolding

61

Query Unfolding

- Query composition is an important mechanism for writing complex queries.
  - Build query from views in a bottom up fashion.
- Query unfolding "unwinds" query composition.
- Important for:
  - Comparing between queries expressed with views
  - Query optimization (to examine all possible join orders)
  - Unfolding may even discover that the composition of two satisfiable queries is unsatisfiable! (exercise: find such an example).

62

Query Unfolding Example

$$Q_1(X,Y) :- Flight(X,Z), Hub(Z), Flight(Z,Y)$$
$$Q_2(X,Y) :- Hub(Z), Flight(Z,X), Flight(X,Y)$$
$$Q_3(X,Z) :- Q_1(X,Y), Q_2(Y,Z)$$

The unfolding of Q_3 is:

$$Q'_3(X,Z) :- Flight(X,U), Hub(U), Flight(U,Y),$$
$$Hub(W), Flight(W,Y), Flight(X,Z)$$

63

Query Unfolding Algorithm

- Find a subgoal $p(X_1,\ldots,X_n)$ such that p is defined by a rule r.
- Unify $p(X_1,\ldots,X_n)$ with the head of r.
- Replace $p(X_1,\ldots,X_n)$ with the result of applying the unifier to the subgoals of r (use fresh variables for the existential variables of r).
- Iterate until no unifications can be found.
- If p is defined by a union of $r_1, \ldots, r_n$, create n rules, for each of the r's.

64

Query Unfolding Summary

- Unfolding does not necessarily create a more efficient query!
  - Just lets the optimizer explore more evaluation strategies.
  - Unfolding is the opposite of rewriting queries using views (see later).
- The size of the resulting query can grow exponentially (exercise: show how).

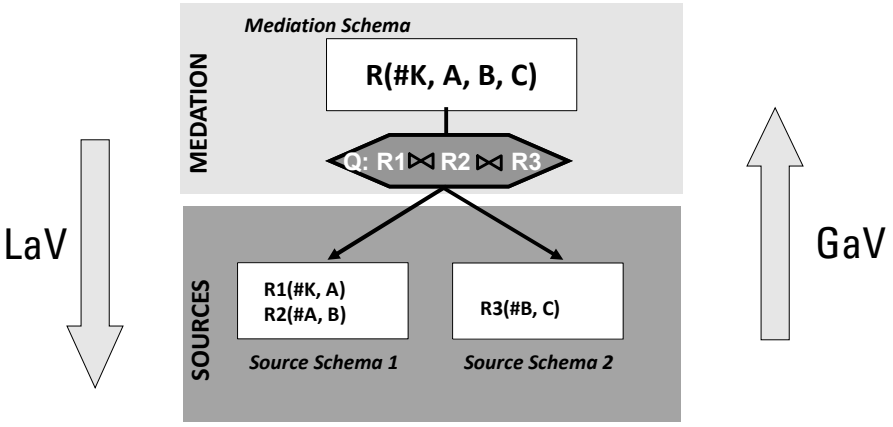
65

Summary

- Source descriptions include:
  - Schema mappings
  - Completeness, access-pattern limitations
  - Data transformations
  - Additional query-processing capabilities
- Schema mapping languages
  - GAV: reformulation by unfolding
  - LAV/GLAV: reformulation by answering queries using views
- Binding patterns and integrity constraints can lead to tricky cases:
  - Recursive rewritings can often address these.

66

Comparaison: GaV vs. LaV



67

67

References

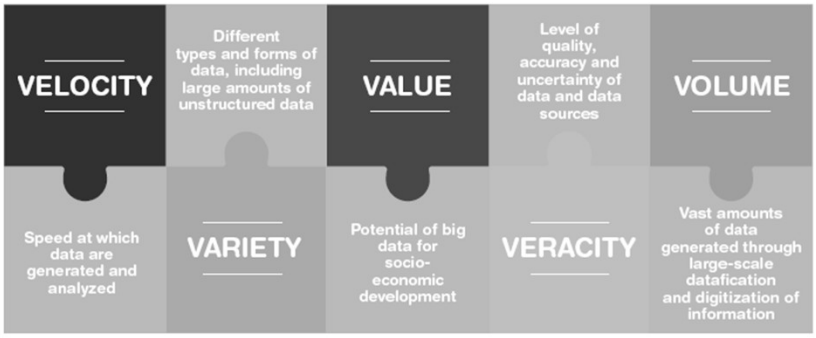
- Ravi Krishnamurthy, Witold Litwin, William Kent: Language Features for Interoperability of Databases with Schematic Discrepancies. SIGMOD 1991, 40-49.
- Laks V. S. Lakshmanan, Fereidoon Sadri, Iyer N. Subramanian: SchemaSQL: An extension to SQL for multidatabase interoperability. ACM TODS, 26(4), December 2001.
- Jeffrey D. Ullman: Information Integration Using Logical Views. ICDT 1997: 19-40.
- M. Lenzerini: Data Integration; A Theoretical Perspective, Proc. ACM Symposium on Principles of Database Systems, 2002.
- Alon Y. Halevy: Theory of Answering Queries Using Views, SIGMOD Record 29(4), December 2000.
- Alon Y. Levy: Answering Queries Using Views: A Survey,VLDB Journal, 2001.
- R. J. Miller, L. M. Haas, M. A. Hernandez. Schema Mapping as Query Discovery. VLDB'00.

68

Addressing Big Data challenges

69

Big data 5'V

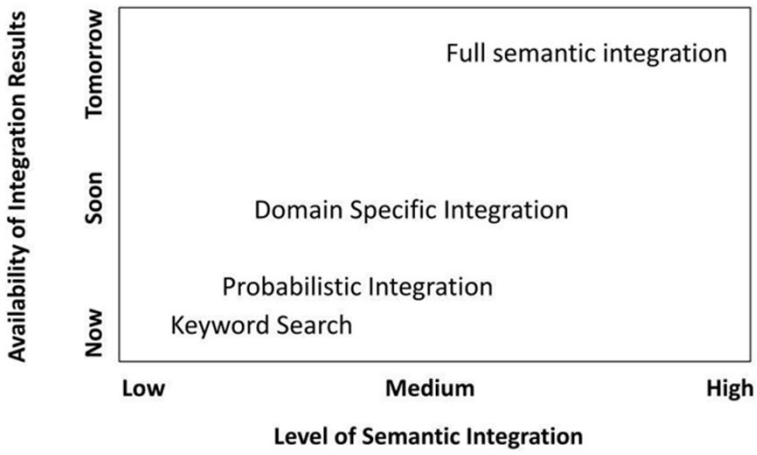


Big data is a term for data sets that are so large or complex that traditional data processing application software is inadequate to deal with them (wikipedia)

70

70

Positioning different approaches



71

WebTables: Keyword Ranking [CHW+08]

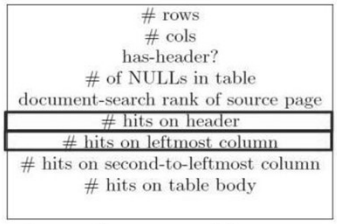
- Goal: Rank tables on web in response to query keywords
  - Not web pages, not individual records
- Challenges:
  - Web page features apply ambiguously to embedded tables
  - Web tables on a page may not all be relevant to a query
  - Web tables have specific features (e.g., schema elements)

34

72

WebTables: Keyword Ranking

- FeatureRank: use table specific features
  - Query independent features
  - Query dependent features
  - Linear regression estimator
  - Heavily weighted features



- Result quality: fraction of high scoring relevant tables

k	Naïve	FeatureRank
10	0.26	0.43
20	0.33	0.56
30	0.34	0.66

35

73

WebTables: Keyword Ranking

- SchemaRank: also include **schema coherency**
  - Use point-wise mutual information (pmi) derived from ACSDb
  - p(S) = fraction of unique schemas containing attributes S
  - pmi(a,b) = log(p(a,b)/(p(a)*p(b)))
  - Coherency = average pmi(a,b) over all a, b in attrs(R)

- Result quality: fraction of high scoring relevant tables

k	Naïve	FeatureRank	SchemaRank
10	0.26	0.43	0.47
20	0.33	0.56	0.59
30	0.34	0.66	0.68

36

74

Why is traditional integration no longer enough?

Deep Web at Scale

Empirical study estimates **~25 million million** deep web sources hidden behind behind HTML forms — content larger larger than the surface web.

647K forms in 25M-page sample

Schema Explosion Explosion

Google Base has **10,000+ item types** and and **~100,000 unique schemas** — "database "database design by the masses" creates creates unbounded heterogeneity.

50+ attributes just for 'vehicle' 'vehicle'

Modeling Everything Everything

On the web, data spans **all of human knowledge**. Domains overlap in complex complex ways — a single mediated schema schema is infeasible and brittle.

Cyc: 20 years, limited success

Keyword Queries

Web users pose **keyword queries**, not structured SQL. Systems must route queries to relevant sources without a global schema.

Requires query routing

75

Traditional DI → PAYGO Architecture

Traditional Data Integration

Single Mediated Schema
Manually designed for one domain

✕ **Exact Schema Mappings**
Semi-automated, verified by experts

Structured Queries
SQL / XQuery over mediated schema schema

✕ **Deterministic Answers**
Complete knowledge assumed upfront

NE W

PAYGO

Schema Clusters
Repository of schemata organized by topic

Approximate Mappings
Probabilistic, between any pair of sources

Keyword Queries + Routing
Reformulate + route to relevant sources

Heterogeneous Ranking
Best-effort ranked answers under uncertainty

76

Two ideas that define PAYGO

01 **Pay-As-You-Go Integration**

Start with little semantic glue.
Answer queries as best possible with what's known.
Continuously improve mappings using automated methods and user feedback.

- Automated schema matching bootstraps
- Implicit & explicit user feedback refines
- Value of Perfect Information (VPI) guides

02 **Uncertainty at All Levels**

uniform formalism to model uncertainty in data, mappings, and queries — so uncertain decisions propagate cleanly across the system.

- Uncertainty in the underlying data
- Uncertainty in semantic mappings
- Uncertainty in query interpretation

77

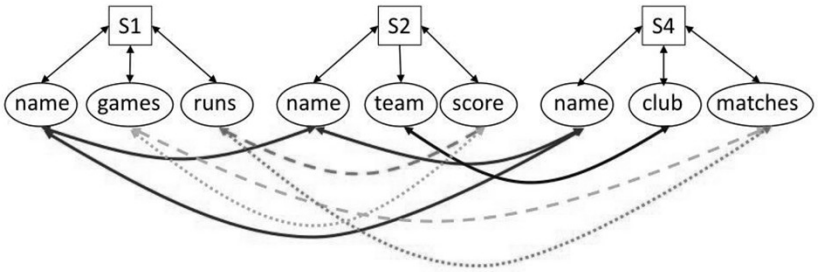
Dataspace Approach [FHM05, HFM06]

- Motivation: SDI approach (as-is) is infeasible for BDI
  - **Volume**, **variety** of sources → unacceptable up-front modeling cost
  - **Velocity** of sources → expensive to maintain integration results
- Key insight: **pay-as-you-go** approach may be feasible
  - Start with simple, universally useful service
  - Iteratively add complexity when and where needed [JFH08]
- Approach has worked for RDBMS, Web, Hadoop …

37

78

Probabilistic Mediated Schemas [DDH08]

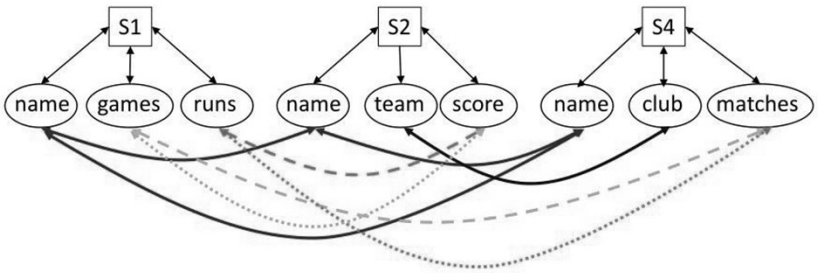


- Mediated schemas: automatically created by inspecting sources
  - Clustering of source attributes
  - **Volume**, **variety** of sources → uncertainty in accuracy of clustering

38

79

Probabilistic Mediated Schemas [DDH08]



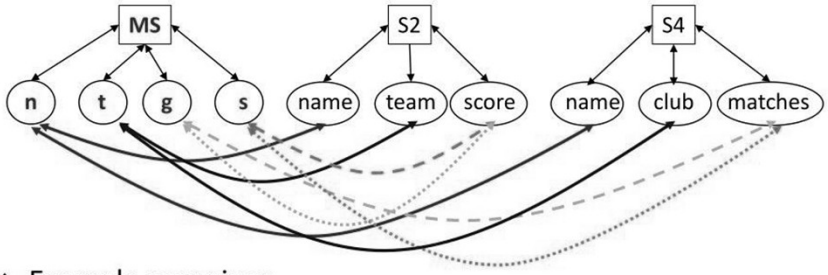
- Example P-mediated schema
  - M1({S1.games, S4.matches}, {S1.runs, S2.score})
  - M2({S1.games, S2.score}, {S1.runs, S4.matches})
  - M = {(M1, 0.6), (M2, 0.2), (M3, 0.1), (M4, 0.1)}

39

80

Probabilistic Mappings [DHY07, DDH09]

- Mapping between P-mediated and source schemas



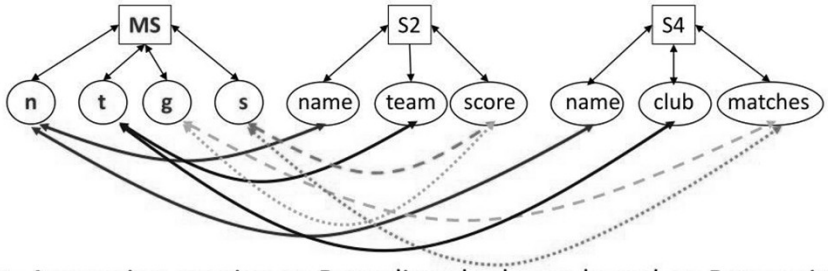
- Example mappings
  - G1({**MS.t**, S2.team, S4.club}, {**MS.g**, S4.matches}, {**MS.s**, S2.score})
  - G2({**MS.t**, S2.team, S4.club}, {**MS.g**, S2.score}, {**MS.s**, S4.matches})
  - G = {(G1, 0.6), (G2, 0.2), (G3, 0.1), (G4, 0.1)}

40

81

Probabilistic Mappings [DHY07, DDH09]

- Mapping between P-mediated and source schemas



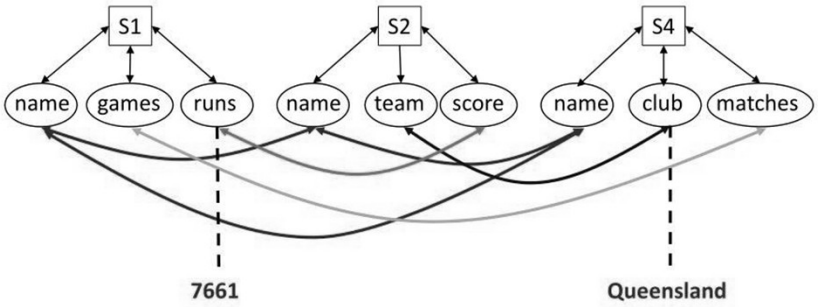
- Answering queries on P-mediated schema based on P-mappings
  - By table semantics: one mapping is correct for all tuples
  - By tuple semantics: different mappings correct for different tuples

41

82

Keyword Search Based Integration [TJM+08]

- Key idea: information need driven integration
  - Search graph: source tables with weighted associations
  - Query keywords: matched to elements in different sources
  - Derive top-k SQL view, using Steiner tree on search graph

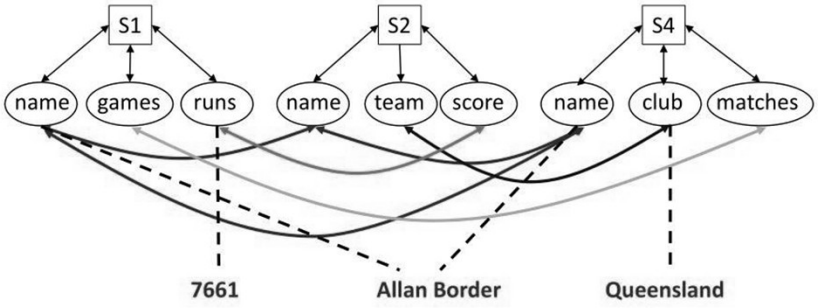


42

83

Keyword Search Based Integration [TJM+08]

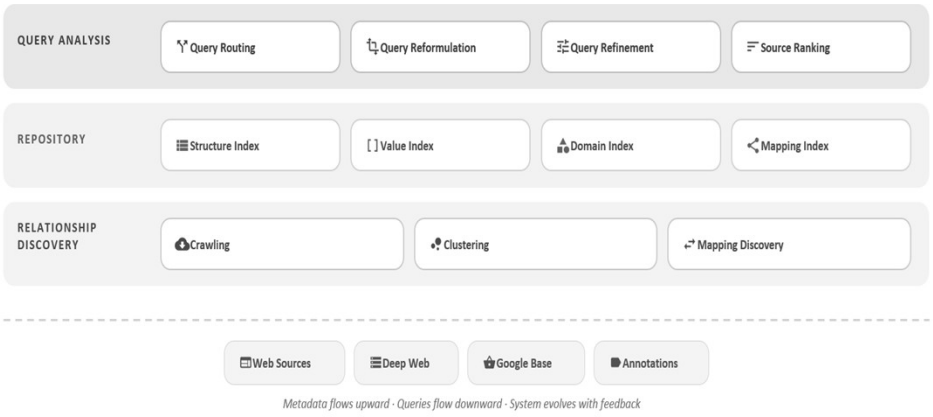
- Key idea: information need driven integration
  - Search graph: source tables with weighted associations
  - Query keywords: matched to elements in different sources
  - Derive top-k SQL view, using Steiner tree on search graph



43

84

System Architecture



85

References

- [CHW+08] Michael J. Cafarella, Alon Y. Halevy, Daisy Zhe Wang, Eugene Wu, Yang Zhang:
  WebTables: exploring the power of tables on the web. PVLDB 1(1): 538-549 (2008)
- [DDH08] Anish Das Sarma, Xin Dong, Alon Y. Halevy: Bootstrapping pay-as-you-go data integration systems. SIGMOD Conference 2008: 861-874
- DDH09] Anish Das Sarma, Xin Luna Dong, Alon Y. Halevy: Data Modeling in Dataspace Support Platforms. Conceptual Modeling: Foundations and Applications 2009: 122-138
- [HFM06] Alon Y. Halevy, Michael J. Franklin, David Maier: Principles of dataspace systems. PODS 2006: 1-9
- [TJM+08] Partha Pratim Talukdar, Marie Jacob, Muhammad Salman Mehmood, Koby Crammer, Zachary G. Ives, Fernando Pereira, Sudipto Guha: Learning to create data-integrating queries. PVLDB 1(1): 785-796 (2008)

86